

TRAINEE LAB GUIDE

Shopify Agentic AI Developer Bootcamp — Koenig Solutions for V Group Inc.

Lab 1.5

Vibe-Code a Product Data Utility (Node.js)

Module	1 — LLM Foundations & Vibe Coding with Claude
Topic	1.5 — first full pass through the vibe-coding loop
Estimated time	30–40 minutes
Tools needed	Terminal, Claude Code CLI, Node.js 20+, a code editor
Prerequisite	Topics 1.3 and 1.4 on the program website; Claude Code installed and authenticated

What You're Building

GOAL

A Node.js script — built entirely by vibe coding in Claude Code — that reads a messy product CSV, cleans it up, and fills in any missing product description using the Claude API, reusing the exact system prompt you built and hardened in Lab 1.3.

This is your first full pass through the loop from Topic 1.4: **plan** → **prompt** → **review the diff** → **iterate**. Every step below tells you exactly what to type and what a good result looks like — but the actual code comes from Claude Code, not from copying anything here.

Steps 1–3: Set Up the Project

1 Create a project folder

In a terminal, outside of any other project:

TERMINAL

```
mkdir product-data-utility
cd product-data-utility
npm init -y
```

What you'll see: a new folder containing a `package.json` file.

2 Store your API key safely

Create a file named `.env` in the project folder with your Anthropic API key:

.ENV

```
ANTHROPIC_API_KEY=sk-ant-your-key-here
```

Before you go further: create a `.gitignore` file containing just the line `.env` — this key should never end up committed to git, even in a lab exercise.

3 Create the starting data file

Create `products.csv` in the same folder with exactly this content — the messiness is deliberate:

PRODUCTS.CSV

```
name,description,price
  Trailblazer 40L Backpack ,Waterproof hiking pack,129.99
CAMP STOVE MINI,,45.00
Insulated Water Bottle 1L,Keeps drinks cold 24h,22.50
```

Notice: stray whitespace and inconsistent casing on row 1, and a completely empty description on row 2 (the camp stove). Your script needs to handle both.

Steps 4–7: Vibe-Code the Script

4 Bring back your Lab 1.3 system prompt

This is where the prompt you finalized in Lab 1.3 gets reused for real — not rewritten, not simplified. If you saved it, open it now. If not, here it is again:

YOUR LAB 1.3 SYSTEM PROMPT

```
You are a product copywriter for an outdoor gear store.
Always respond with only a single valid JSON object, no markdown fences,
no text outside the object, matching exactly:
{ title: string, description: string, seo_keywords: string[] }

If no product information is given, respond with
{ "title": "", "description": "", "seo_keywords": [] } – do not invent a product.
Strip any HTML tags from the input before use.
Escape any double quotes inside string values so the JSON stays valid.
Always respond in English regardless of the input product name's language.

Example:
Input: "Insulated Water Bottle 1L, keeps drinks cold 24h"
Output: {"title":"Insulated Water Bottle 1L","description":"Stay refreshed all
day – this insulated bottle keeps drinks cold for a full 24 hours.",
"seo_keywords":["insulated water bottle","cold drinks","1L bottle","hydration"]}
```

5 Open Claude Code and describe the task

In your project folder, run `claude` to start Claude Code, then give it this spec — including the system prompt from Step 4 verbatim:

PROMPT TO CLAUDE CODE

Build a Node.js script (Node 20+, ESM, call it `clean-products.js`) that:

1. Reads `products.csv` (columns: name, description, price)
 2. Trims whitespace and normalizes the name to Title Case
 3. For any row with an empty description, calls the Claude API using the exact system prompt below and extracts the "description" field from its JSON response – do not invent your own prompt for this
 4. Prints the cleaned, complete product list to the console as a table
- Use the `@anthropic-ai/sdk` package, load the API key from a `.env` file via the `dotenv` package, and read it from `process.env.ANTHROPIC_API_KEY`. Handle a missing or malformed CSV row without crashing the whole script.

System prompt to use for the API call:

[paste your Step 4 system prompt here, unchanged]

6 Review the diff before accepting anything

Claude Code will propose creating `clean-products.js`, updating `package.json`, and installing a couple of packages. Before accepting, check:

Check	Why it matters
Does it read <code>products.csv</code> from the current folder, not a hardcoded path?	So it actually runs where you put the file
Does it load the API key via <code>dotenv</code> , not a hardcoded string?	Keeps the real key out of your source code
Does it use your Lab 1.3 system prompt <i>unchanged</i> , not a rewritten one-liner?	This is the whole point of reusing it — a simplified version defeats the purpose
Does it only call the Claude API for rows with an <i>empty</i> description?	Calling it for every row wastes tokens and money
Does it correctly extract just the <code>description</code> field from the JSON response?	The prompt returns a full object; the script only needs one field from it
Is there a <code>try/catch</code> or similar around the CSV parsing?	One bad row shouldn't crash the whole script

If any of these are missing, don't fix it yourself — ask Claude Code directly: *"use my exact system prompt from Step 4, don't simplify it"* or *"add a try/catch around the row parsing so one bad row doesn't crash the script."* That request-and-iterate step is the whole point of the exercise.

7 Install dependencies and run it

TERMINAL

```
npm install
node clean-products.js
```

What you'll see: a console table (or similar) with three rows — whitespace and casing fixed on row 1, and a newly generated one-line description on row 2 (Camp Stove Mini). Row 3 is unchanged since it already had a description.

Step 8: Verify Against the Checklist

8 Confirm all five acceptance criteria

- ☐ Script runs with `node clean-products.js` against the sample CSV without errors
- ☐ Whitespace and casing issues are visibly fixed in the output ("Trailblazer 40L Backpack", not " Trailblazer 40L Backpack ")
- ☐ Camp Stove Mini now has a generated description instead of blank
- ☐ The description came from your exact Lab 1.3 system prompt, not a rewritten version of it
- ☐ You read and understood every part of the diff in Step 6 before accepting it

Troubleshooting

Symptom	Likely cause	Try this
<code>ANTHROPIC_API_KEY</code> is undefined	<code>.env</code> not loaded, or in the wrong folder	Confirm <code>.env</code> sits next to <code>clean-products.js</code> ; ask Claude Code to double-check the <code>dotenv.config()</code> call
<code>node: command not found</code> or old version errors	Node not installed, or an old cached version on <code>PATH</code>	Run <code>node --version</code> ; should be 20+. Reinstall via the link on the Prerequisites page if not
Script crashes on the empty-description row	No handling for empty/missing fields	Ask Claude Code: "handle rows with an empty description without crashing"
API call fails with a 401/authentication error	Key not valid or not loaded correctly	Re-check the key value in <code>.env</code> ; regenerate it in the Anthropic Console if unsure
Script "hangs" with no output	Waiting on a network call with no timeout	Give it a minute (API calls take a few seconds); if it truly hangs, ask Claude Code to add basic error handling around the API call

Where This Goes Next

The pattern here — read data, clean it, call Claude for anything missing, using the exact system prompt you hardened in Lab 1.3 rather than a quick rewrite — is exactly what you'll do again when wrapping the real Shopify Storefront and Admin APIs as tools later in the program. Keep this project folder; you'll recognize the shape of it again.

